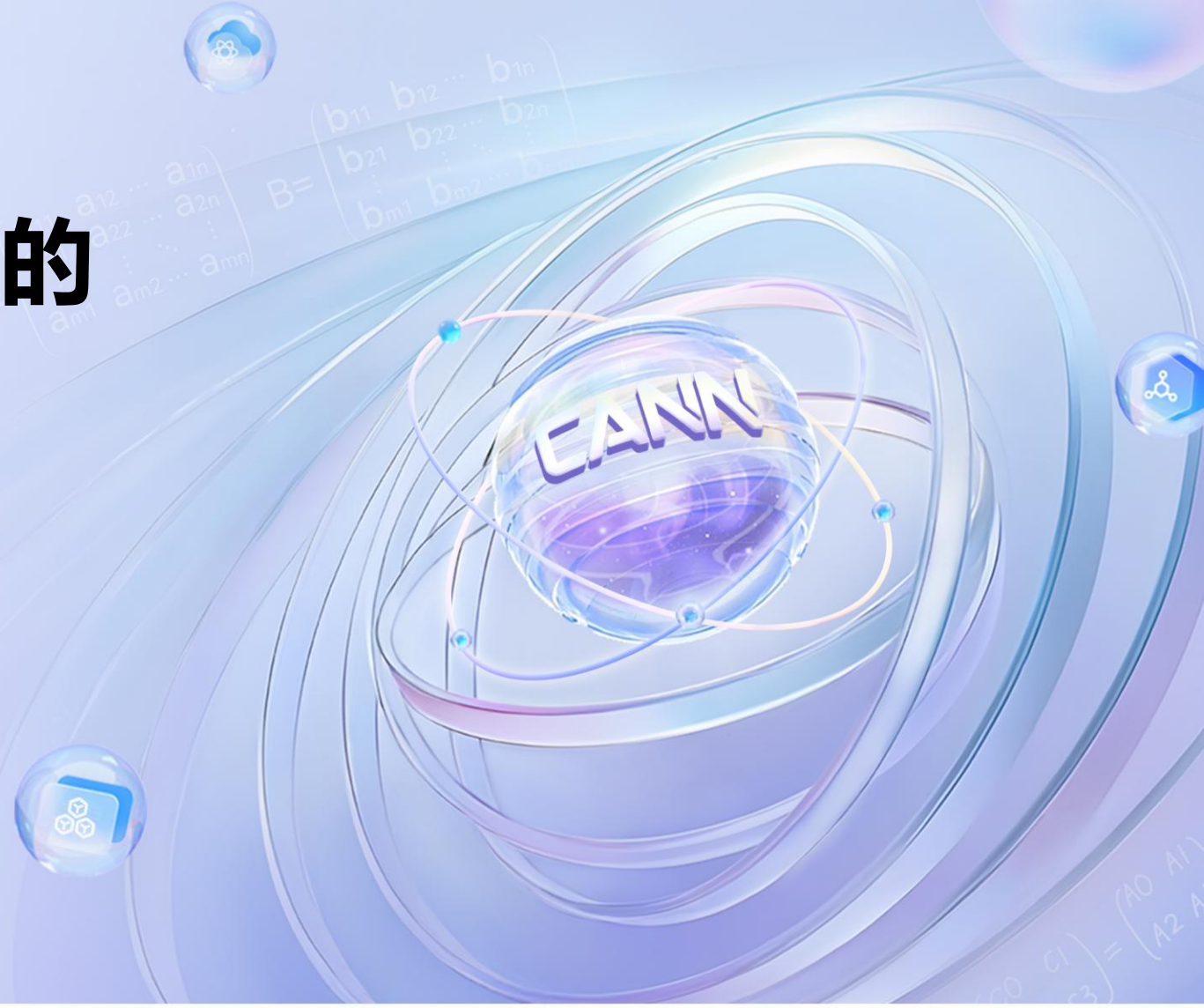
Deepseek-V4.1-flash的MegaKernel融合实践



时间：2026.11.10

CANN

Megakernel：整网部署与性能收益

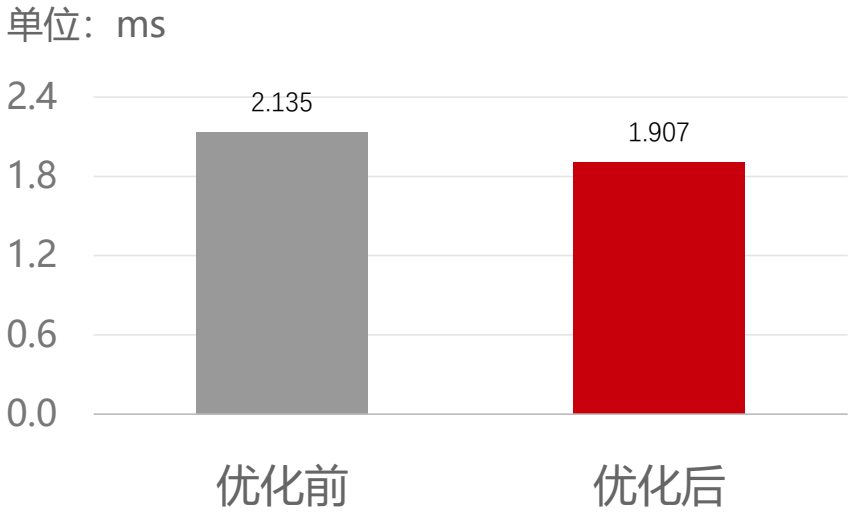
DeepSeek V4.1-Flash ｜ B×S=8，Head 按 TP8 切分

整网部署的五类融合算子

融合算子	覆盖职责
mega-mla	Q/KV 前处理与注意力计算
mega-full-index	完整索引生成
mega-reindex	候选重评分与索引刷新
mega-compress	压缩状态生成与历史更新
attn-epilogue	注意力尾处理与输出投影

整网推理性能优化

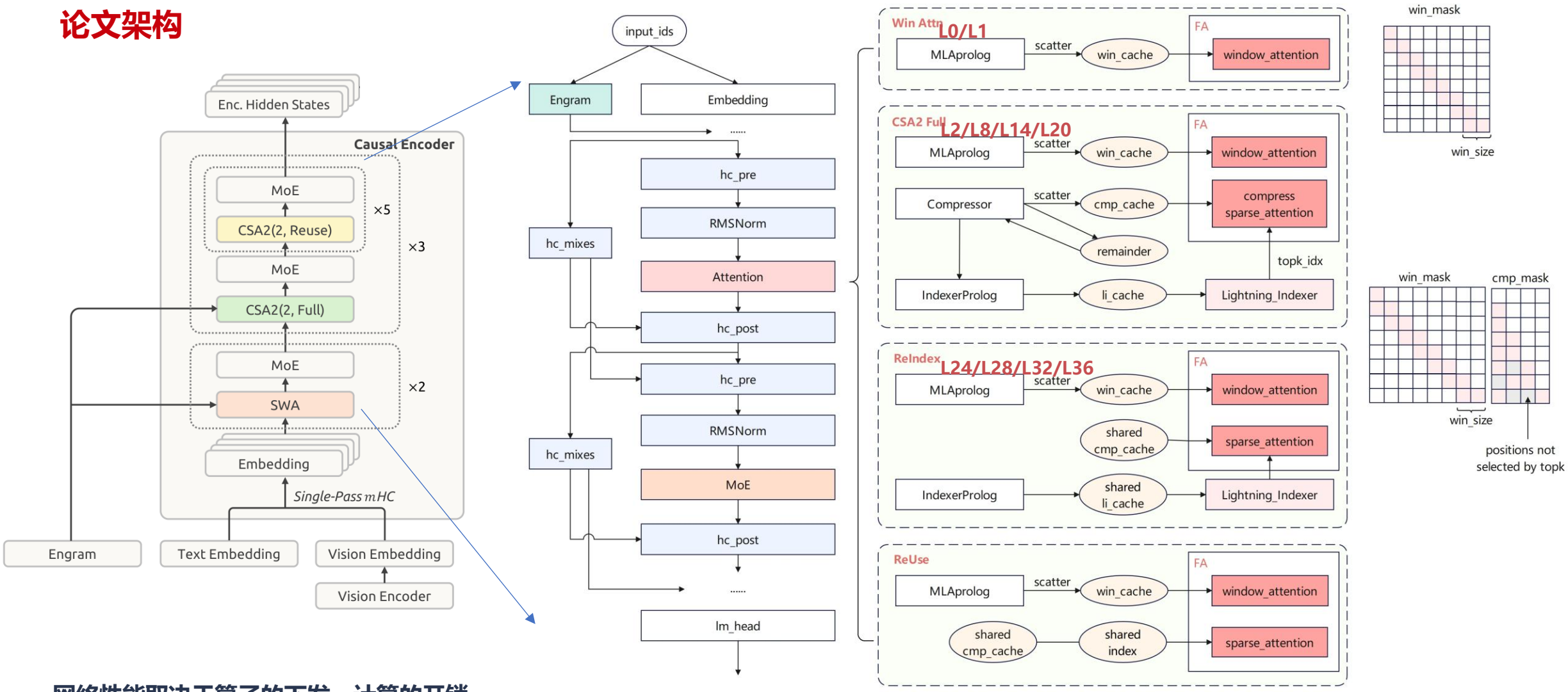
10.68%



五类融合路径共同优化，单次推理整网性能优化10.68%。

DeepSeek V4.1-Flash：网络结构

论文架构



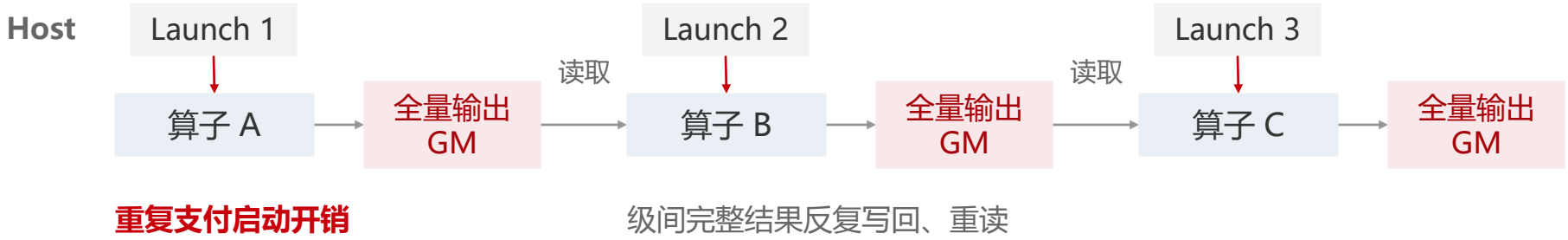
网络性能取决于算子的下发、计算的开销。

在小 Batch Decode 情况下，kernel launch、算子间数据读写、同步等待、核心未充分使用的问题容易增加推理时延。

单算子串行下发的边界开销

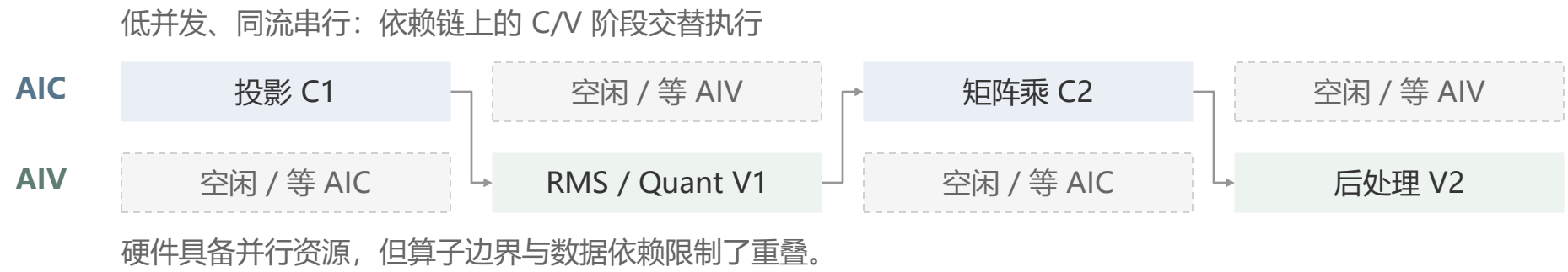
01 多次启动与全量写回 GM

每次下发都有启动开销。
完整输出写回 GM，
下游算子再从GM读取。



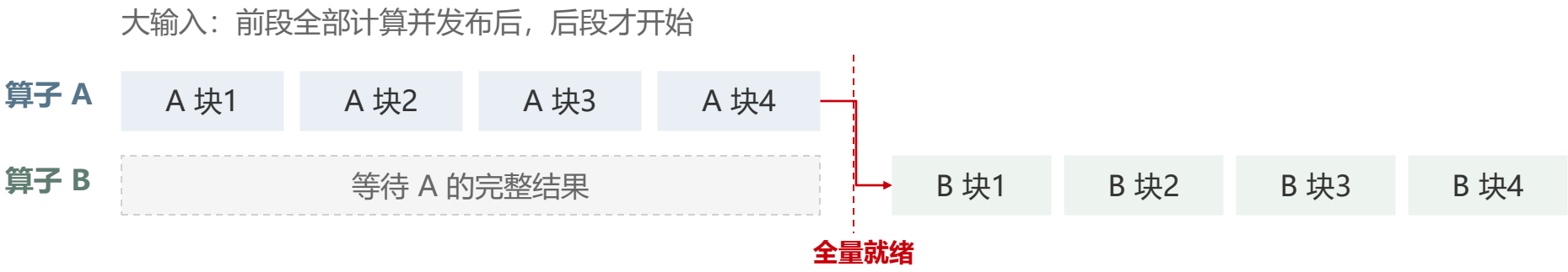
02 C/V 交替空闲

AIC 等 AIV，反向也相同。
低并发时，缺少其他任务
填补空闲区间。



03 全量计算后才能启动下游

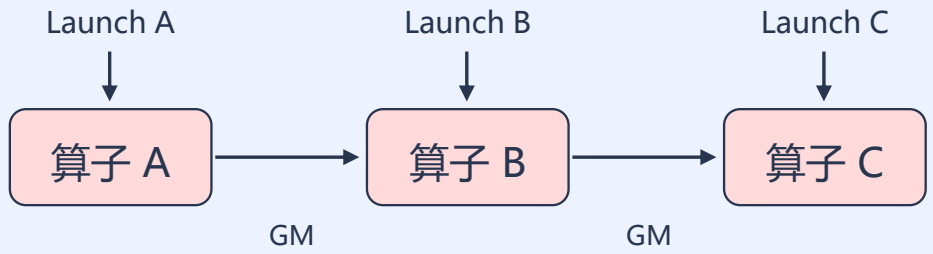
输入越大，前段计算越长。
单kernel输出可能挤出L2空间
，下一个算子的输入无法充分
利用L2



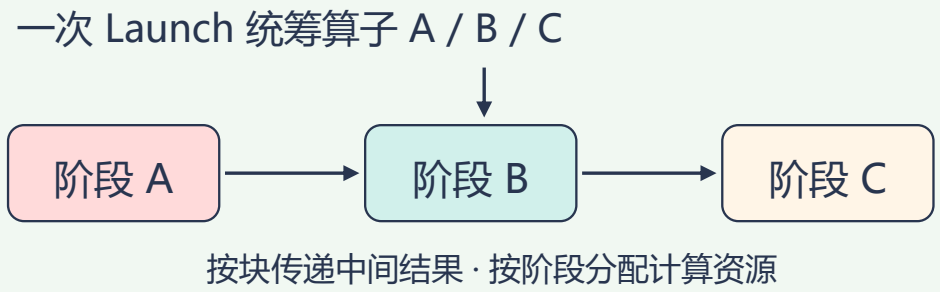
Megakernel：执行方式与收益来源

将具有数据依赖的算子组织到一次设备启动内，统一安排计算、数据搬运和同步。

多个算子分别下发



一个 Megakernel 统一调度



01 数据片上复用

02 CV并行

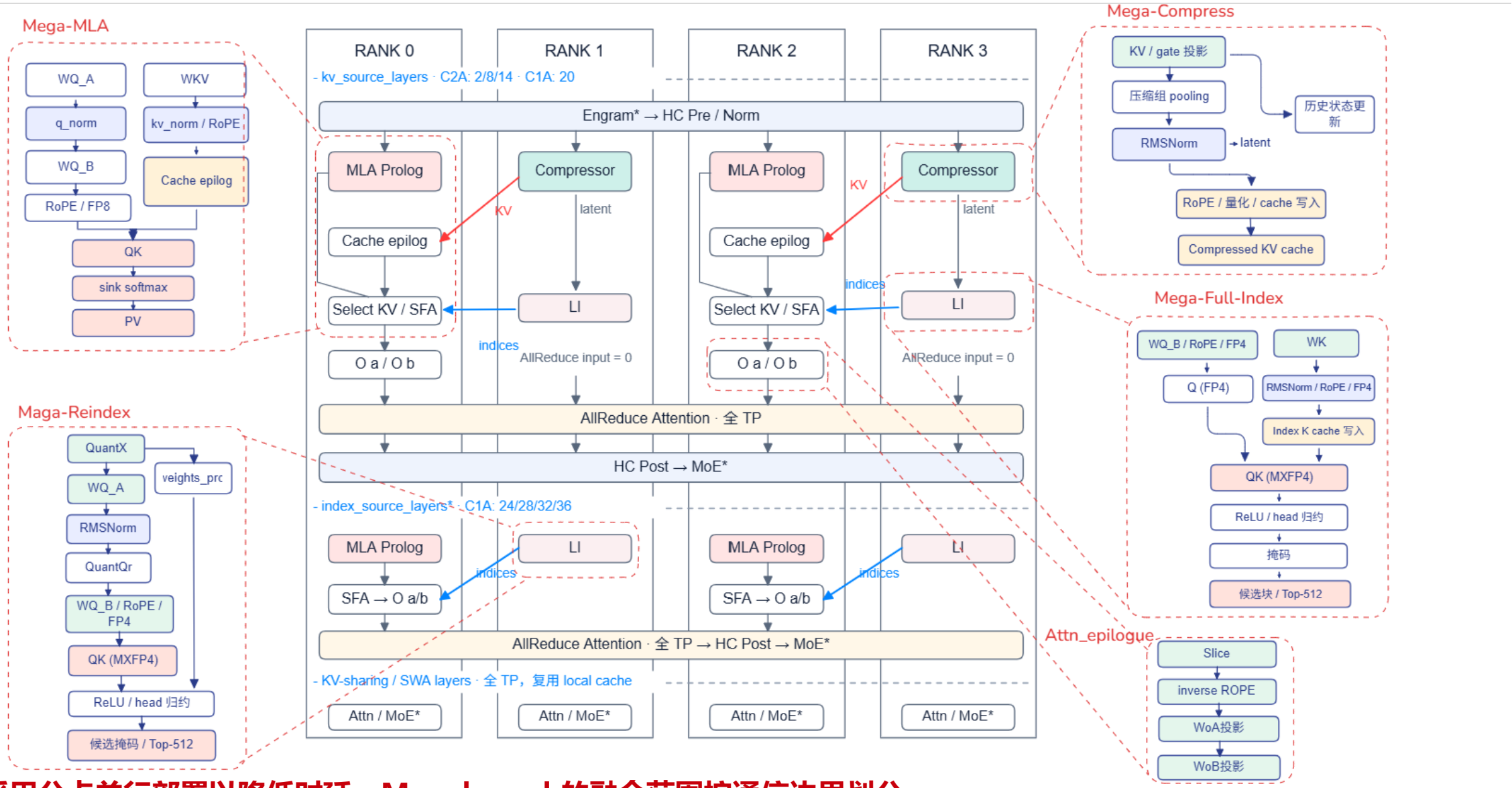
03 减少重复launch

业界实践：Hazy Research 融合 Llama-1B 的单卡 decode 前向计算；
MPK 以设备任务图组织多卡计算与通信。

本次实践根据分卡部署和通信节点，划分 mega_mla、mega_full-index、mega_reindex、mega_compressor，attn_epilogue。

参考：Hazy Research, No Bubbles（2025）；MPK, arXiv:2512.22219。

Megakernel：分卡部署与融合边界



整网采用分卡并行部署以降低时延，Megakernel 的融合范围按通信边界划分。

Mega-MLA：网络位置与融合范围

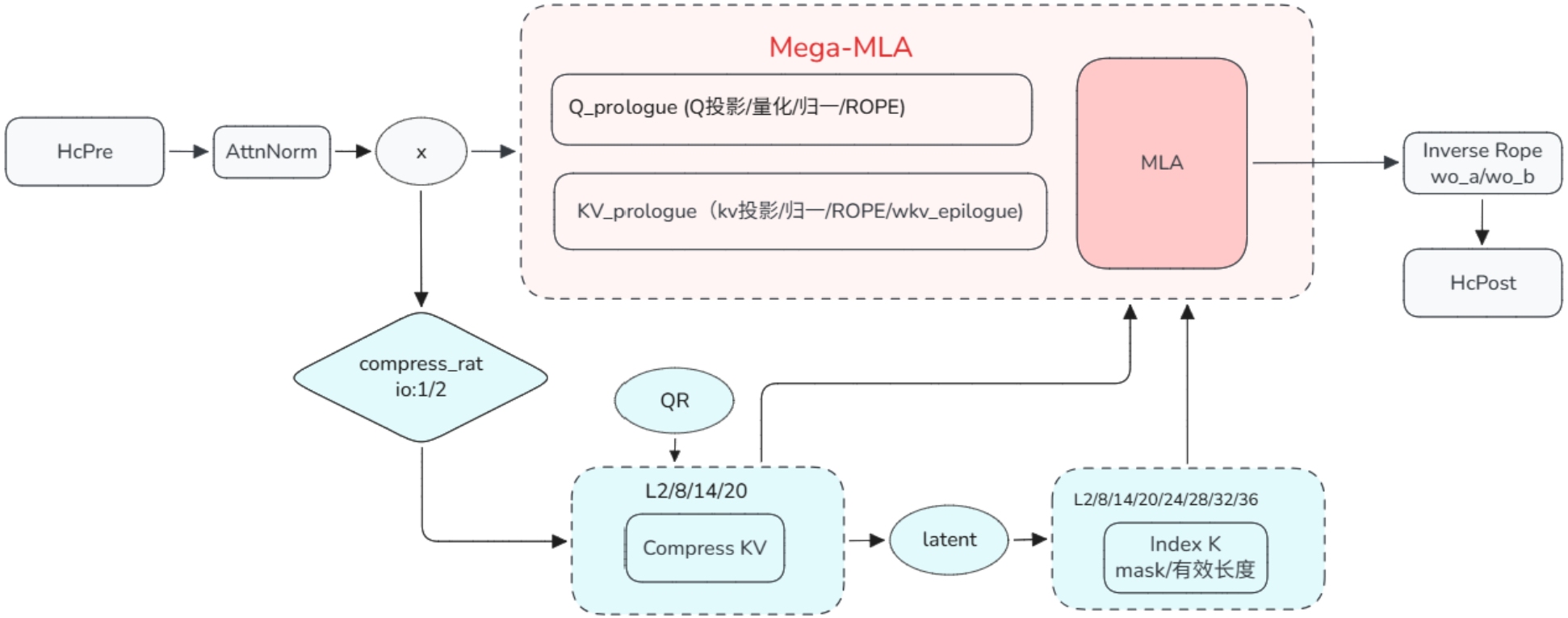
融合范围：HcPre + Norm 之后，从 Q/KV 预处理至 MLA 输出，不含 Compressor 与 Indexer 计算

40层整体结构

- 每层生成自己的Q和Win_KV
- L2/8/14/20为compress源层，生成共享cache
- L2/8/14/20/28/32/36为index层，输出topK位置
- 其他层复用上述几层compress和index的输出

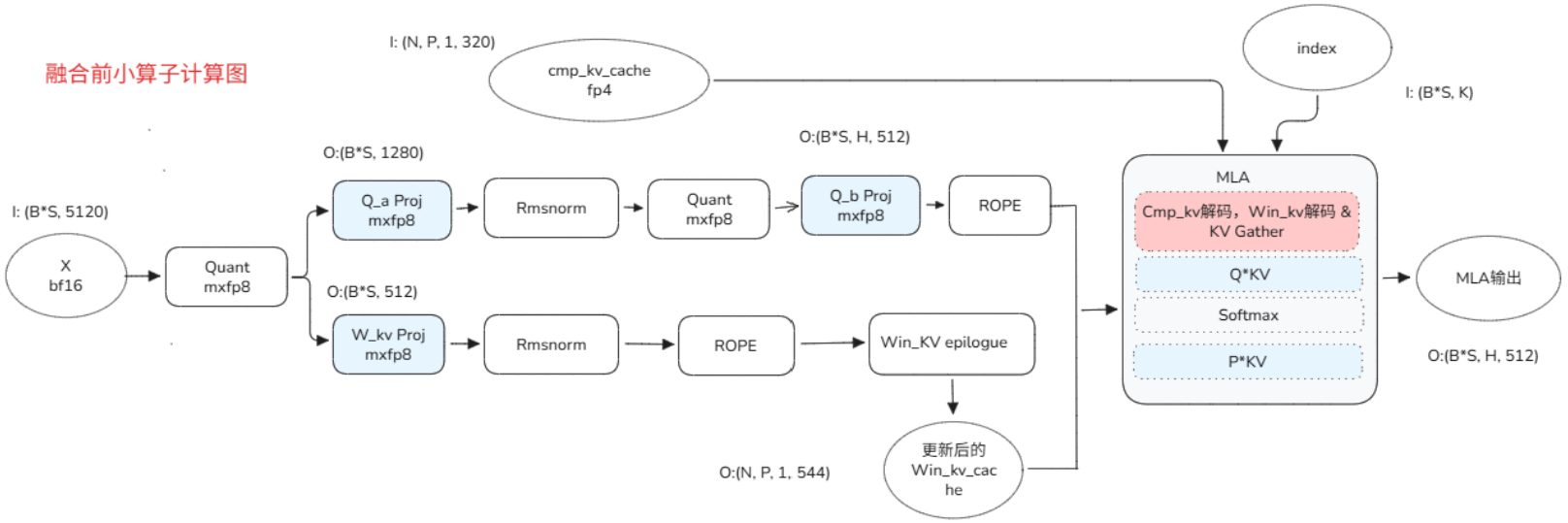
网络部署

- compress源层/index源层存在单层跨卡，prologue和compress/index卡间并行
- 在非compress/index源层以及非L0/L1层部署该算子，共30层



Mega-MLA：分支并行与片内复用

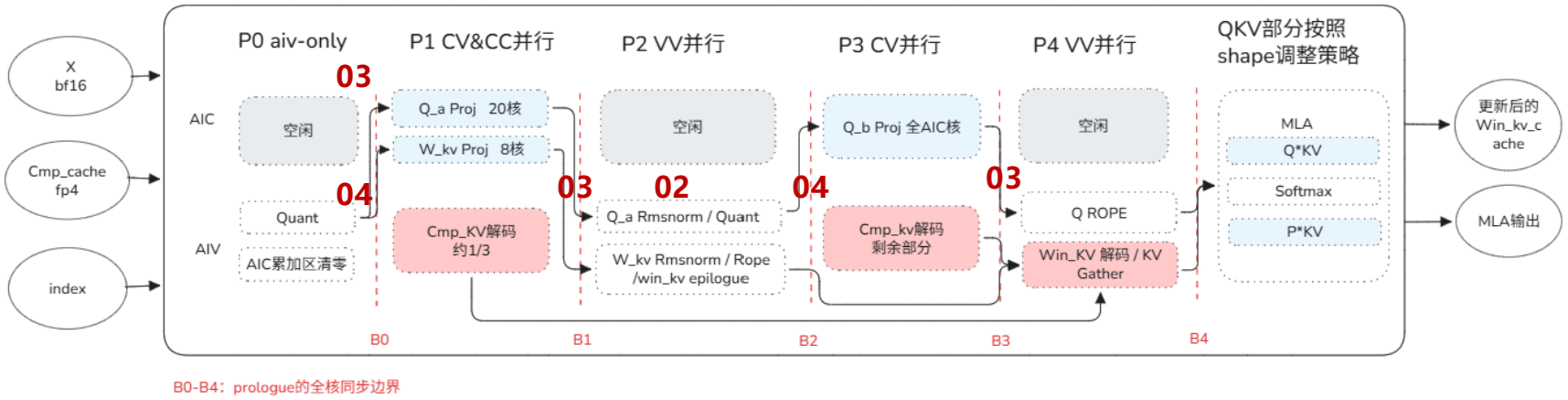
融合前小算子计算图



融合前的瓶颈

01 多次下发，启动开销较大
02 算子间经 GM 中转
03 单阶段难以用满多核
04 AIC / AIV 交替执行

融合后计算逻辑时序图



B0-B4：prologue的全核同步边界

融合后的优化

01 单次下发
11 次下发合并为 1 次

02 片内复用
Q/KV 分支减少 3 次 GM 中转

03 分支共享计算核
P1 共享 AIC，P2 / P4 共享 AIV

04 时序调整
压缩 KV 解码提前至 P1 / P3

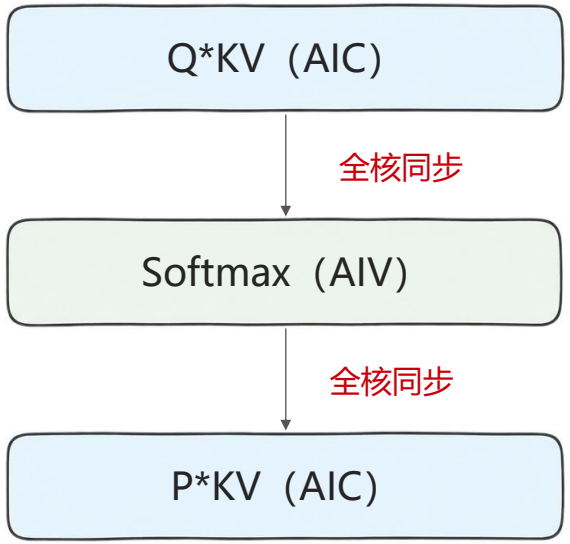
Mega-MLA：切分策略与 C/V 流水

Q × KV、Softmax、P × KV 根据 shape 选择切分与执行方式

分段串行：CV间GM中转

适用于较小 T/H 场景

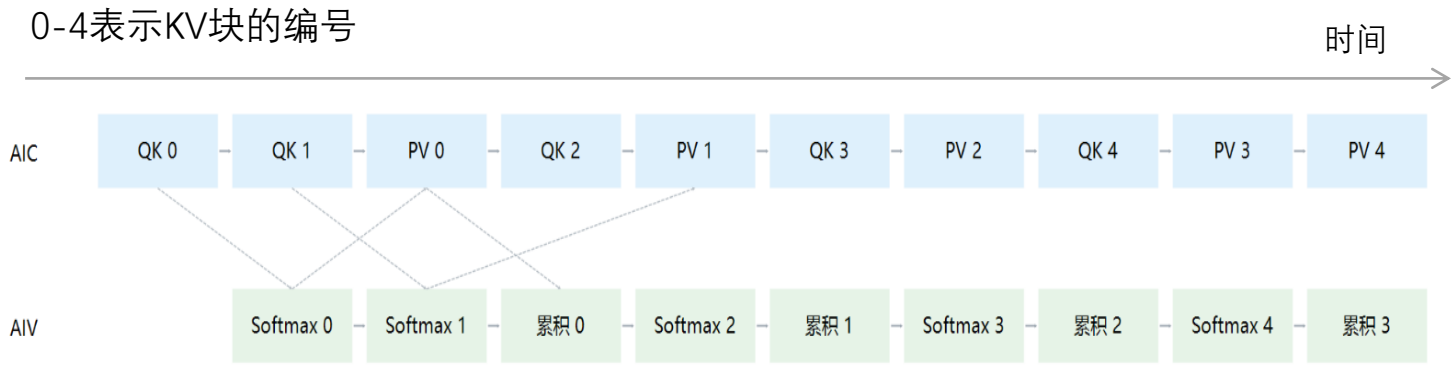
H8 / H16，或小 T 的 H64：AIC 单轮可完成计算，且 Softmax 归约轴参与分核。



分块并行：CV块间并行交错

适用于 T/H 较大、可按 KV 分块的场景

H64 / T16 / KV640 / 32 核：按 H、T 分核。AIC 单核 H=32、T=1，AIV 单核 H=16、T=1。KV 每 128 个元素分块循环，使用 online softmax 补偿块间指数。



Mega-MLA：性能收益

性能收益点

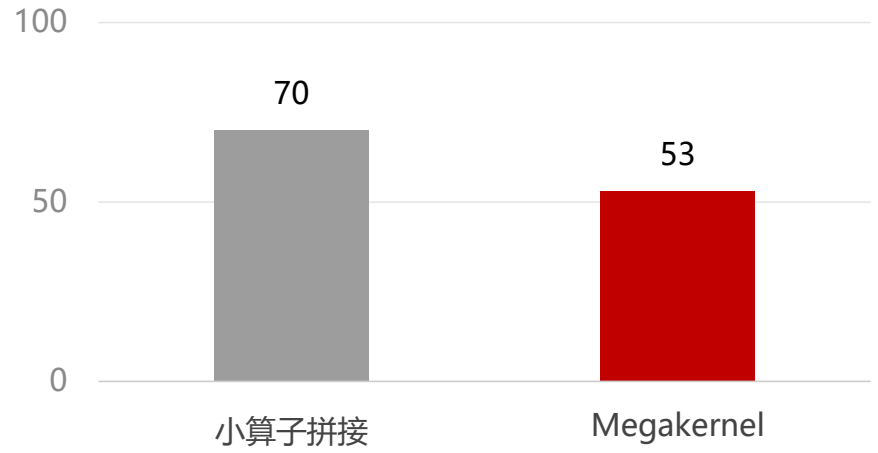
优化策略	实现方法	收益原理
单次启动	11次 kernel 下发 合并为1次 Megakernel	减少启动与调度开销
访存优化	片内连续计算，数据片内复用， 或CV间数据直搬	减少 GM 往返与重复读取
调度并行	Q/KV 分支共享 C/V 核 压缩 KV 解码与两段投影 矩阵计算并行	提升负载均衡 减少资源空闲
计算重组	按规格择优选择 online softmax	计算优化依赖重组

低时延场景AIC的核利用率提升7%， AIV核利用率提升35%

性能测试结果

单位：μs

性能提升 24.3%



参数项	实际测试规格
负载规模	T=B*S=8, KV长度=640
计算维度	H=64，D=512
环境配置	Ascend950DT

核利用率 = 有任务区间核时 ÷ 总可用核时

Mega-Full-Index：网络位置与融合范围

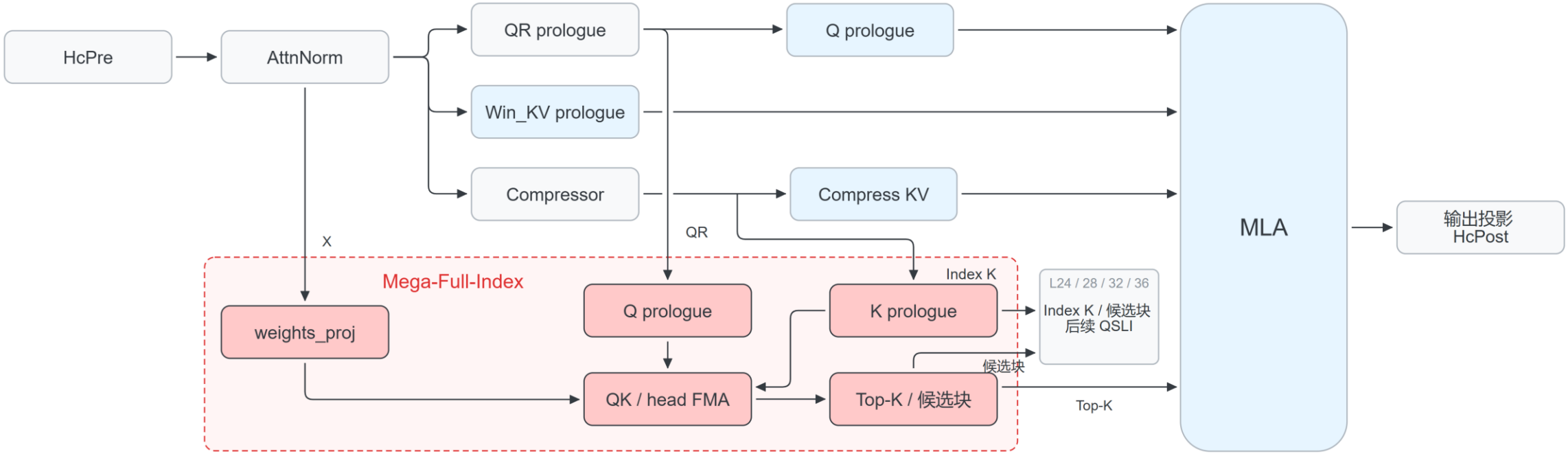
L20 作为索引源层，生成 Index K 与候选集合，为当前层稀疏注意力及后续重索引层提供输入。

Full-Index 概述

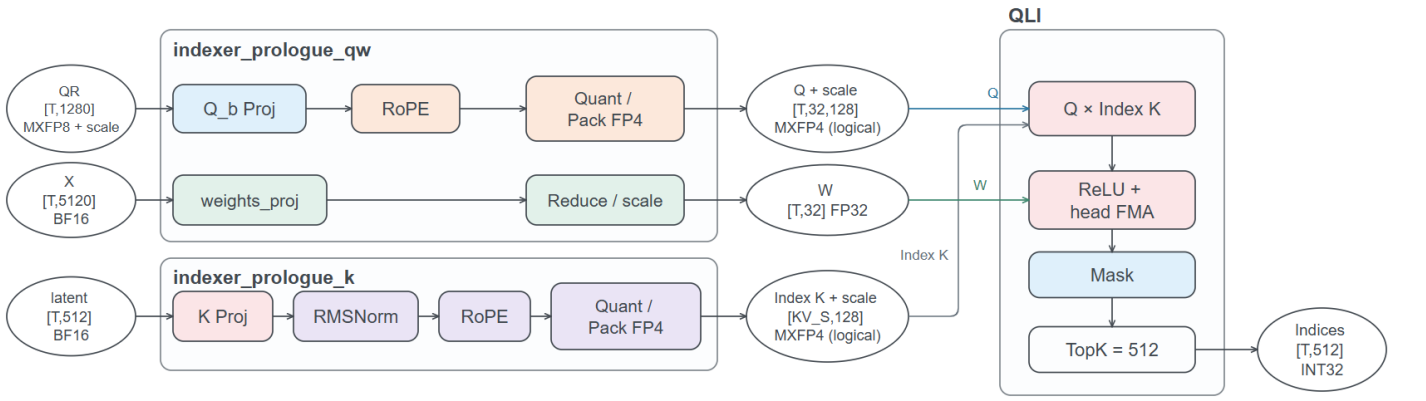
- 生成 Indexer Q、K 及逐头权重，更新并共享 Index K cache。
- 对完整可见索引范围计算相关性得分，输出 Top-512 结果，供当前层稀疏注意力使用。
- 根据同一组得分生成候选块集合，供 L24、L28、L32、L36 重新打分。

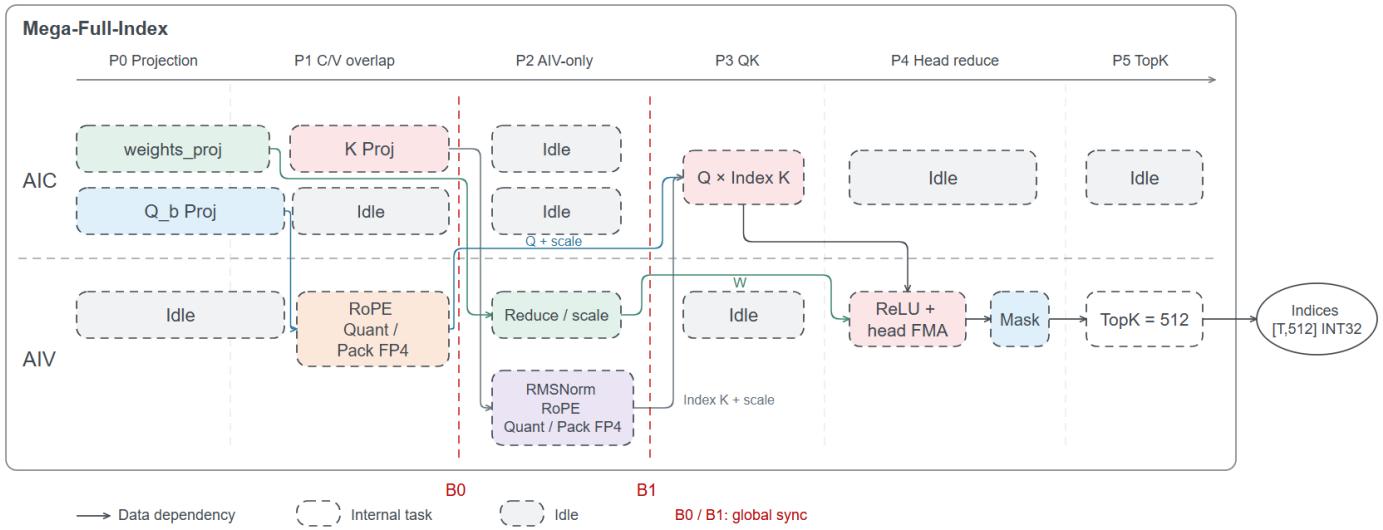
融合边界

- 融合 Prologue QW、Prologue K 与 QLI。
- 覆盖 Q/W/K 投影、向量后处理、得分计算与 Top-K 计算。



Mega-Full-Index：联合切分与并行调度





核心优化策略：

根据数据依赖关系及计算量大小，综合设计最优的切分及调度策略。

① 依赖分析

Prologue QW 中的 Q_b Proj、 weights_proj 计算以及 Prologue K 中的 K Proj 计算是三个相互独立的矩阵运算，具备并行条件。
三条投影分支的后续 vector 运算之间也可以并行处理。

② 切分设计

根据三个矩阵运算量的大小，设计切分及分核策略如图所示。其中，Q_b Proj 运算量最大，其次是 weights_proj，K Proj 的运算量最小。优先为 Q_b Proj 和 weights_proj 各分配 16 个 AIC，在 weights_proj 计算完成后下发 K Proj 任务。

③ 调度设计

Q_b Proj 任务优先下发，与 weights_proj，K Proj 计算并行。
Q_b Proj 后续的 vector 任务可以与 K Proj 计算并行。
weights_proj 后续 vector 运算可以与 K Proj 后续的 vector 运算并行下发。
三条计算分支完成后，进一步下发 QLI 运算相关任务。

Mega-Full-Index：性能收益

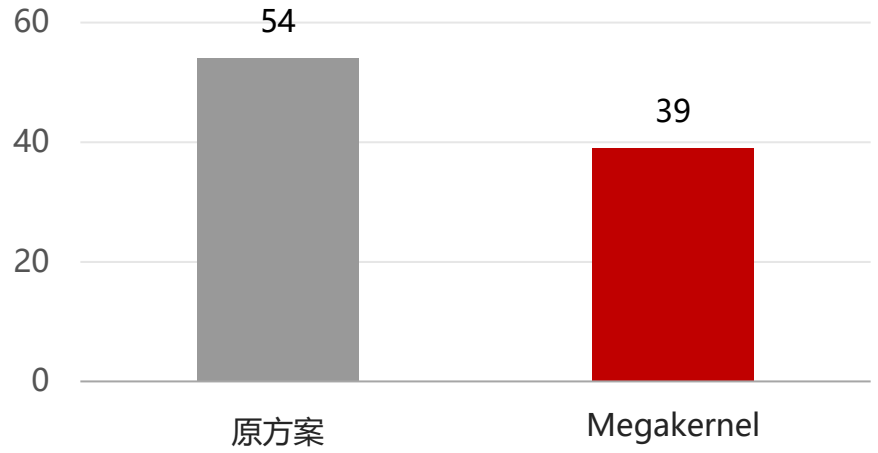
性能收益点

优化策略	实现方法	收益原理
单次启动	Prologue QW / K 与 QLI 3 次启动合并为 1 次	减少启动与调度开销
调度并行	Q/W/K 三条预处理计算分支 并行调度。	提升负载均衡程度， 减少资源空闲
联合分核	按 Q/W/K 分支的运算量 联合设计切分与核组分配	保证单核最佳切分，并有效 提升负载均衡

性能测试结果

单位：μs

性能提升约 27.8%



参数项	实际测试规格
负载规模	T=8，KV_S=8192
计算维度	Nq=32，D=128，TopK=512
环境配置	Ascend950DT

Mega-ReIndex：网络位置与融合范围

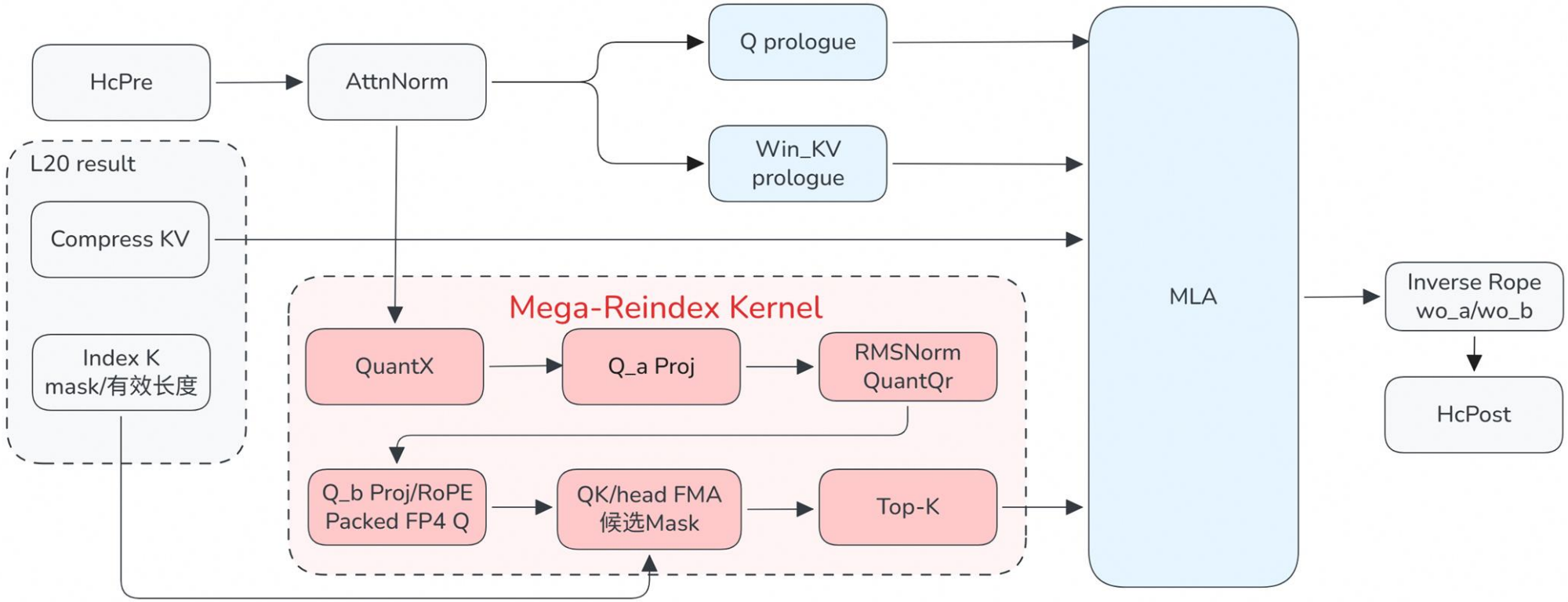
L24 / L28 / L32 / L36 复用 L20 的 Index K 与候选 mask，重新计算得分并生成 Top-K

生成TopK，作为稀疏注意力前置输入

- L24、L28、L32、L36结构相同
- 使用L20共享 Index K 与候选Mask；
- 重新生成Q，计算候选得分，输出Topk；

多卡并行；单卡融合

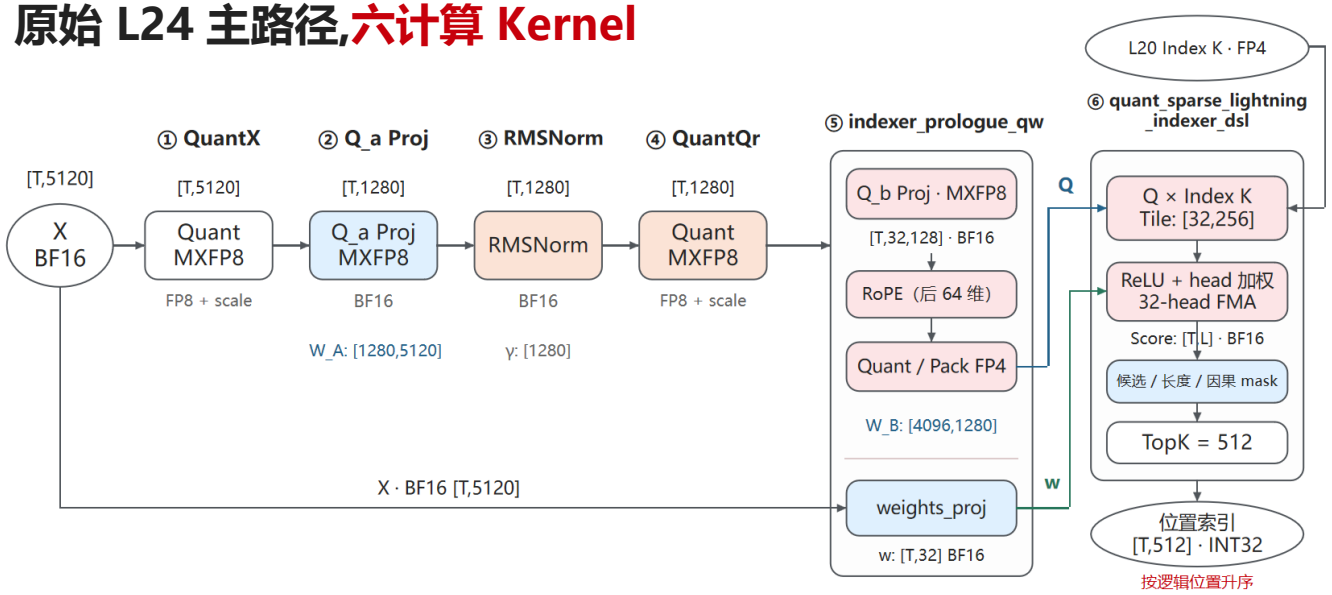
- 多卡并行：将无数据依赖的分支在多卡部署，多卡并行；
- 单卡融合：多任务并行；数据片内传递；C/V核并行；搬运计算并行；



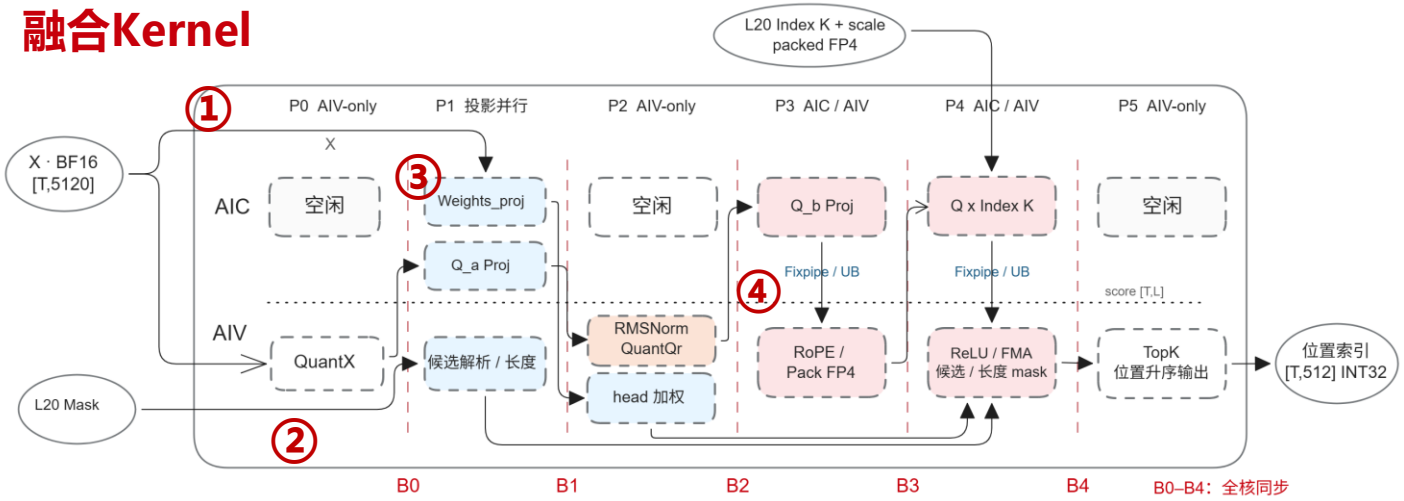
图示为 L24 网络结构：从 hc_pre 到 hc_post

Mega-ReIndex：分支并行与片内传递

原始 L24 主路径,六计算 Kernel



融合Kernel



① 单次启动

6次 kernel调度启动合并为1次 Megakernel 启动，优化kernel launch耗时。

② 内存压缩

候选 mask：BOOL [T, KV_S] 按 8-token 块压缩为 UINT8 [T, ceil(KV_S / 8)]

③ 并行调度

资源无依赖，CC\CV分核计算并行

- Cube计算：Q_a Proj、 weight_proj
- Vector计算：候选 mask、PA 页长度读取

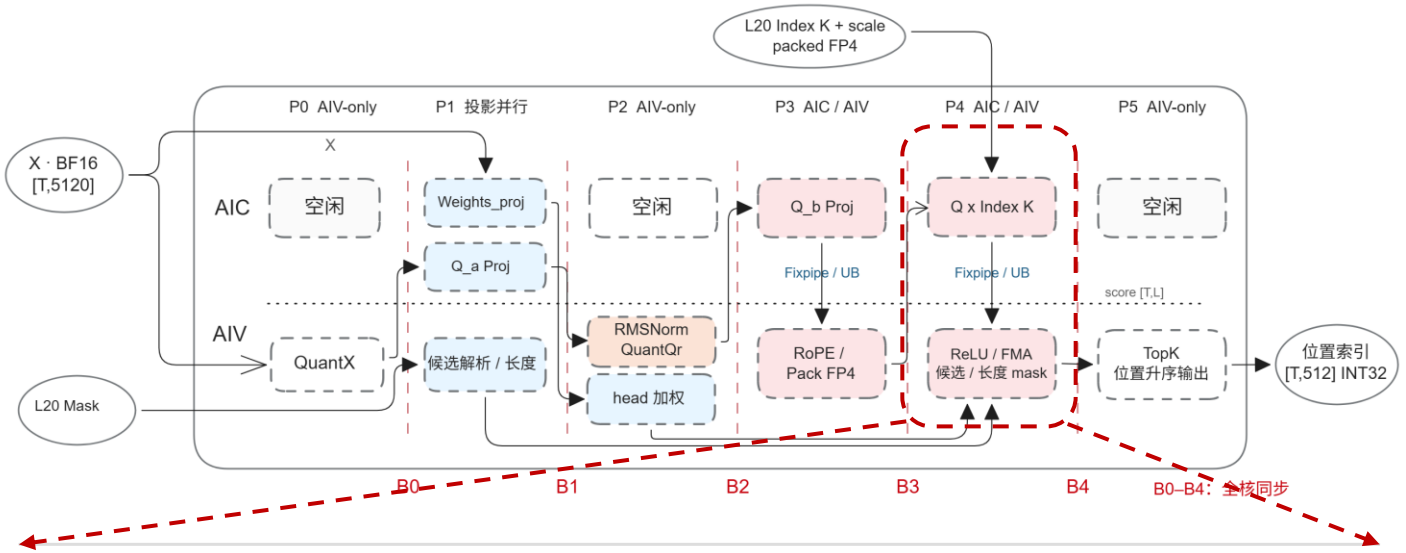
④ 访存优化

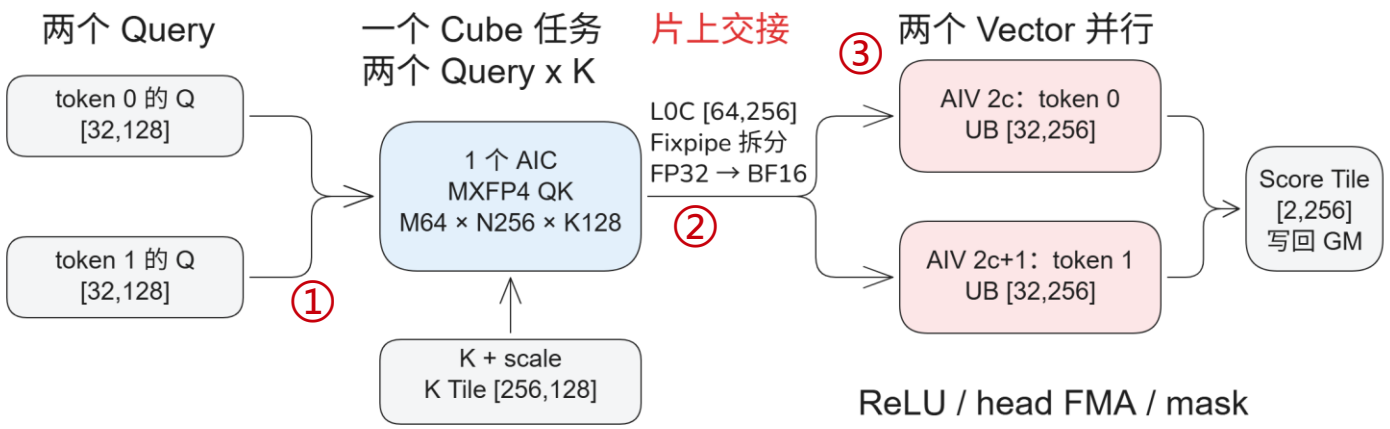
片内计算传递，免逐写回 GM；

- Vector数据传递：UB内计算数据传递
  Norm 计算结果以Tile粒度在UB中QuantQr
- Cube-Vector数据传递，
  Q_b Proj →RoPE / FP4，使用L0C → UB通路

Mega-ReIndex：QK 复用与 C/V 流水

QK Pair 复用：2 路 token、每路 32 heads 合并为 M64，复用 K Tile 进行 QK 计算





① 减少重复搬运

两个Token共用K Tile
减少重复搬运与任务调度

② 数据片内复用

QK结果从L0C->UB
避免数据写回GM

③ 拆分2路Vector并行计算

L0C->UB启动两路Vector
自动拆分两路token数据

④ CV流水优化

CV Tile 块间并行交错
切块间轮转并行，CV间直传，不经过GM通路

Mega-ReIndex：性能收益

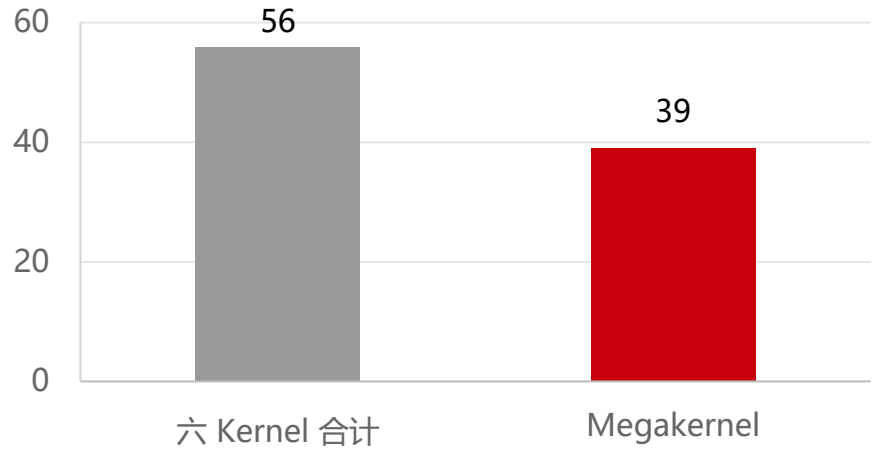
性能收益点

优化策略	实现方法	收益原理
单次启动	6 次 kernel 下发 合并为 1 次 Megakernel	减少启动与调度开销
调度并行	统一切分与核组分配， 支持 C/C、C/V 分核并行	改善负载均衡 减少资源空闲
访存优化	片内传递计算结果， 减少写回 GM	减少 GM 往返与重复读取 提升访存效率
内存压缩	候选 mask 每 8-token 块 使用 1 个 UINT8 标记	64 KiB 降至 8 KiB 存储量减少 87.5%

性能测试结果

单位：μs

性能提升 30.43%



参数项	实际测试规格
负载规模	B=1，S=8，T=8，KV_S=8192
计算维度	32 heads，D=128，TopK=512，压缩比=1
运行环境	Ascend 950DT

Megakernel：共性收益与泛化挑战

共性收益

灵活分核：低时延场景下，相比多流控核，可按节点调整并行方式和核数，改善负载均衡
时序调整：穿插彼此无依赖的计算，减少串行等待
减少搬运：通过线性化计算、片上传递和 UB 复用，减少 GM 访问

挑战	根因与表现	工程影响
01 运算结构泛化	网络变化会改变投影、稀疏模式、Cache 结构及算子依赖。 C/V 比例与关键路径也会变化。	既有分核、片上驻留和同步顺序可能失效。 最优融合拓扑需重新评估。
02 最优模板泛化	同一计算图中，T/H/K、核数与 dtype 改变切块和缓冲容量。 小 T 与大 T 需要不同分核策略。	Shape × Tile × Buffer × 核数带来模板组合增加， 不利于泛化场景支持

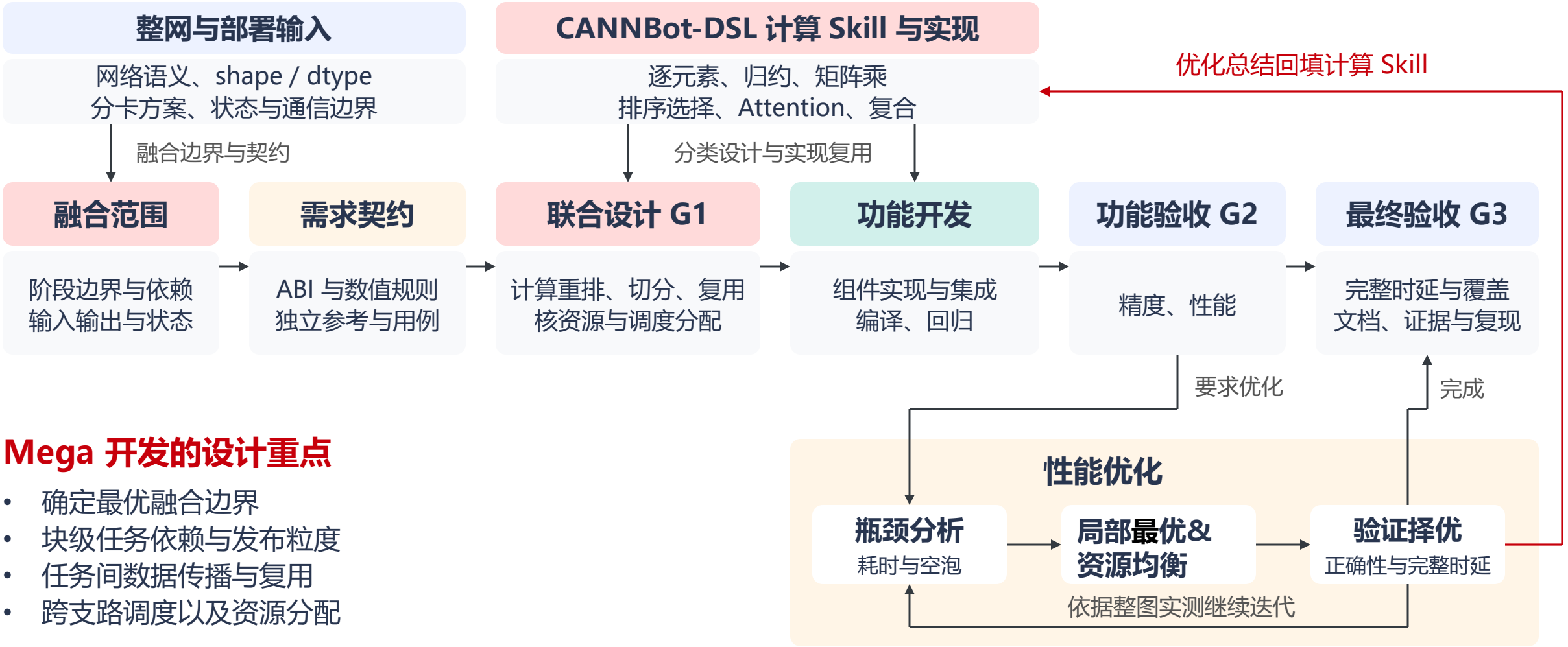
场景边界与开发方式

本次实现面向低时延业务，围绕具体网络结构与 shape 组合设计，融合与调度策略需按场景验证。
借助 CANNBot-DSL 与 Agent，加快方案实现和遍历，并依据实测结果选择更优方案。

本次实践只是基于dsv4.1-flash的一种megakernel的实践，当前收益也属于阶段性成果。我们仍在持续探索更多优化方向，争取进一步降低整网推理时延

CANNBot：Megakernel 开发与优化工作流

下一步：基于 CANNBot 构建工作流，联合设计融合范围、任务图与执行调度



Mega 开发的设计重点

- 确定最优融合边界
- 块级任务依赖与发布粒度
- 任务间数据传播与复用
- 跨支路调度以及资源分配

CANNBot-DSL： https://gitcode.com/cann/cannbot-dsl

Thank you.

社区愿景：打造开放易用、技术领先的AI算力新生态

社区使命：使能开发者基于CANN社区自主研究创新，构筑根深叶茂、跨产业协同共享共赢的CANN生态

Vision: Building an Open, Easy-to-Use, and Technology-leading AI Computing Ecosystem

Mission: Enable developers to independently research and innovate based on the CANN community and build a win-win CANN ecosystem with deep roots and cross-industry collaboration and sharing.

上CANN社区获取干货

关注CANN公众号获取资讯

https://gitcode.com/cann